SYSTEM MAP · EVIDENCE · NEXT STEPS

How Report Navigator turns a PDF into an answer.

A visual guide to the complete pipeline: what each component does, why it is here, how answers are checked, and where the system still needs stronger evidence.

Python + Streamlit | SQLite-backed report store | Gemini answer path | Current numeric-answer scope

Scope note. The current answer contract returns an integer count or `NOT_FOUND`. It does not identify people in photos, explain graphs visually, or return open-ended names. Page images are rendered for citation viewing but are not sent to Gemini.

DATA

What is in the current workspace?

The counts below describe the supplied report and its local ingestion, not answer accuracy.

Gold question mix

50 supplied questions · 45 numeric answers · 5 expected `NOT_FOUND`



This chart summarizes the gold file's answer types. It does not show the app's score.

Local ingestion snapshot

Each bar is scaled to this list's maximum; units differ.

PDF pages	272
Search chunks	7,352
Parsed rows	6,333
Pages with rows	170
OCR pages	0

Snapshot from the local SQLite database; another PDF will produce different counts.

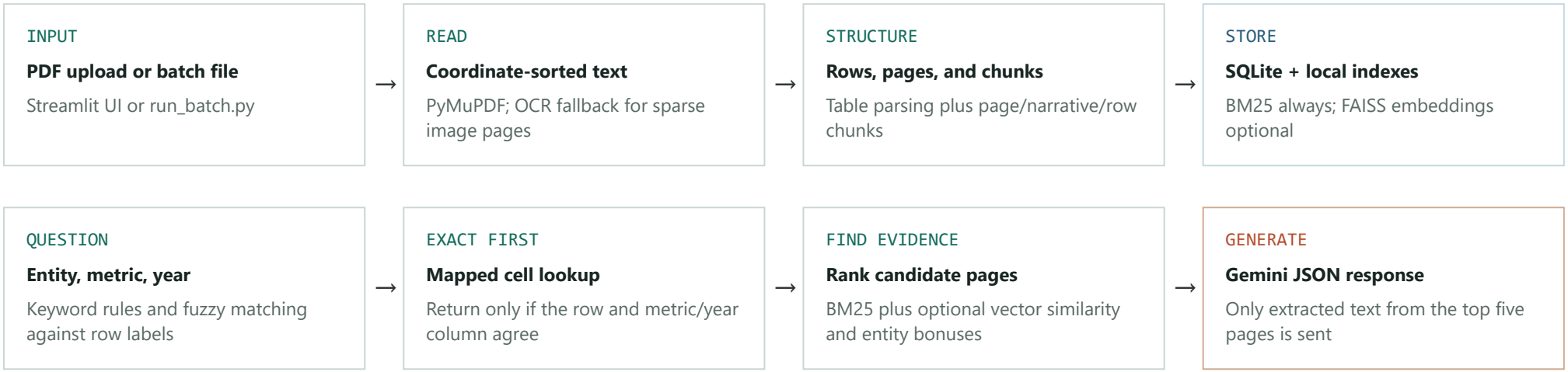
Answer benchmark score: not available

No saved batch output or computed exact-match/page-accuracy report was found in the workspace. No accuracy percentage is claimed here.

FLOW

Two paths share one evidence store.

Ingestion prepares searchable text and rows. Question answering first tries exact structured data, then retrieves pages and asks Gemini when needed.



The validator checks the JSON shape, numeric value, and citation page. It does not yet prove that the number is under the correct table column.

What happens when a stage cannot proceed?

- Missing optional embeddings or FAISS: continue with BM25.
- Entity not found in text or stored row labels: abstain before calling Gemini.
- No API key: return a low-confidence fallback instead of fabricating a value.

- Malformed output or a number/page that fails validation: retry once with the same page context, then show the failure.

PDF ingestion: recover text before interpreting it.

The ingestion boundary converts each page into reusable evidence. It is not a vision-understanding pipeline.

---

1. Extract page text and layout

`src/ingest/pdf_loader.py` reads words with coordinates and sorts them top-to-bottom, then left-to-right. The page text layer is preferred because it preserves exact digits better than OCR. PyMuPDF also renders page images for the citation viewer.

Why: some report PDFs store words in a reading order that does not match the visual page. Coordinates help restore readable lines.

---

2. Use OCR only as a fallback

`src/ingest/ocr.py` runs Tesseract when a page has fewer than 50 native text characters and contains an embedded image. OCR output becomes plain searchable text.

Limit: OCR can confuse digits and does not preserve the table grid or graph relationships. A scanned chart may yield labels, not a trustworthy series of values.

---

3. Extract table rows, with cautious headers

`src/ingest/table_extract.py` tries PyMuPDF `find_tables()` on number-dense pages, then falls back to numeric rows from sorted text lines. Row values are retained even when the header cannot be mapped confidently into named columns.

Why this matters: the fallback row is useful for retrieval and Gemini context, but a list of values is not enough for a safe deterministic answer. The question may target a particular year, category, or sub-column.

---

4. Chunk and index the document

- **Page chunks** preserve the broad page context.
- **Narrative chunks** are about 800 characters with 120 characters of overlap.
- **Table-row chunks** include the row label, title, and either mapped values or an ordered value list.

SQLite stores documents, page text, row data, chunks, and QA history. BM25 is built for every document; optional BGE embeddings and FAISS cover narrative and table-row chunks.

Question answering: deterministic first, Gemini second.

This is a retrieval-augmented generation system. Gemini does not search the entire PDF by itself; the app chooses page text and sends that context.

Context extraction

`entity.py` identifies common metrics, years, supported subject phrases, and entity labels found in stored table rows. RapidFuzz helps match spelling variants. This is rule-based question parsing, not a general-purpose language understanding layer.

Hybrid page retrieval

`hybrid_search.py` scores chunks with a small in-repository BM25 implementation, optionally adds FAISS similarity, aggregates scores by page, and boosts pages containing the entity and its table row. Singular/plural token variants help align question wording with report wording.

The top five candidate pages are passed forward by default. The prompt includes at most 6,000 characters from each page. If the relevant page or its headers do not make this cut, Gemini cannot use them.

Gemini request, response, and trace

`llm_client.py` makes a standard-library HTTP request to the OpenAI-compatible endpoint in `.env`. The answer trace in the UI shows endpoint, model, request messages,

status, and raw response/error. The API key is excluded, but page text is included because that is what the provider receives.

Current local configuration uses Gemini; source defaults fall back to a Groq endpoint if no override is provided. Model timeouts and model availability depend on provider/network state.

Output validation and citation

The active answer schema is integer string, page number, and confidence, or `NOT_FOUND`. The validator strips commas and confirms the number appears somewhere in a candidate page. It may move the page citation to another candidate page containing that number.

Not yet validated: that the number is at the intersection of the requested entity row and the intended year/category column.

Multimodal boundary. The model request contains text messages only. The saved page PNG is shown as a citation preview; it is not attached to Gemini. Questions about faces, chart shapes, or visual table layout need a separate multimodal request path.

Technology choices and alternatives.

The current stack favors local, inspectable parts and a small deployment footprint.

JOB	USED	WHY IT FITS	ALTERNATIVES AND TRADE-OFFS
Application	Python	One language for PDF parsing, search, UI, and batch work.	Java or C# can add stricter application structure; Python has a faster document/ML iteration path.
PDF text/layout	PyMuPDF	Fast word coordinates, page rendering, and table detection.	pdfplumber is useful for table inspection; pypdf is lighter but has less layout/render support.
Scanned text	Tesseract + Pillow	Local and low-cost for pages with no usable text layer.	PaddleOCR/EasyOCR can improve some scripts/layouts; cloud OCR adds cost and sends documents to a provider.
Structured store	SQLite	Single file, no service to operate, enough for one report collection.	PostgreSQL is stronger for multi-user production concurrency; it does not solve header interpretation.
Keyword search	Custom BM25	Always available, local, exact names/years matter.	rank-bm25 is a library alternative; OpenSearch adds advanced querying and service operations.
Vector search	FAISS + Sentence Transformers (optional)	Local semantic matching without a vector server.	Chroma/pgvector simplify metadata workflows; hosted vectors add cost and data transfer.
Embeddings	BAAI/bge-small-en-v1.5	Small cached English model; stays local.	Other local models may handle multilingual reports better; API embeddings simplify install but transmit text.

JOB	USED	WHY IT FITS	ALTERNATIVES AND TRADE-OFFS
Answer model	Gemini via OpenAI-compatible HTTP	Endpoint/model can change through environment config; no OpenAI SDK dependency.	Google GenAI SDK exposes native multimodal and Gemini-specific features; that is the better route for page-image questions.
User interface	Streamlit	Fast Python upload, chat, progress, and batch UI.	FastAPI + React offers finer UI and API boundaries at more implementation/hosting cost.
Validation	Python + regex	Simple and transparent for an integer answer contract.	Pydantic/JSON Schema help for richer data structures, but do not prove the answer uses the right table cell.

Core requirements: PyMuPDF, Streamlit, python-dotenv, RapidFuzz, pandas. Optional requirements: Sentence Transformers, FAISS, pytesseract, Pillow, pdfplumber. Tesseract also needs its system executable.

MEASURE

Metrics: what is measured and what is missing?

`evaluate.py` compares a batch output against gold answers. No score file exists in this workspace yet.

METRIC	DEFINITION IN THIS PROJECT	WHAT IT CANNOT TELL US
Exact match	Exact answer-string matches / all gold questions.	No partial credit; a correct number with formatting mismatch can fail.
Page accuracy	For numeric gold items answered numerically, predicted page is in the gold page set.	Page may be correct while the wrong column/number was chosen. NOT_FOUND and missing answers are excluded from this denominator.
NOT_FOUND precision	Correct abstentions / all predicted abstentions.	Does not measure how many answerable questions were missed.
NOT_FOUND recall	Correct abstentions / all gold NOT_FOUND items.	Can look good if the model over-abstains; read with precision and exact match.

How a full evaluation should be read

Exact match covers the answer; page accuracy covers citation location; NOT_FOUND precision/recall cover abstention behavior. Together they are useful but still miss table-cell correctness and retrieval misses.

Recommended next metrics: retrieval Recall@5 (did the gold page reach Gemini?), answer coverage, row-and-column evidence accuracy, latency/cost per question, and confidence calibration on held-out questions.

EVIDENCE

How sure are we that an answer is right?

Confidence should come from evidence checks and evaluation, not from a model's self-rating alone.

A	B	C	D
Value exists	**Page is eligible**	**Cell is correct**	**Score holds at scale**
Validator sees the number somewhere on a candidate page.	Citation is limited to pages sent to Gemini.	A reviewer aligns entity, year, measure, and category headers.	Gold evaluation measures exact match and page accuracy.

The current validator proves A and part of B. It does not prove C. D is not available until a full batch is run and scored.

- Gemini confidence is not calibrated. An answer with confidence 1.0 can still be wrong.
- Code assigns 0.95 to deterministic cell answers; that is a heuristic, not a measured probability.
- LLM-returned `NOT_FOUND` confidence is capped at 0.5. The earlier entity-absent path currently uses 0.8.
- Manual source inspection should compare the full table header hierarchy with the entity row, not just search for the number on the page.

ROADMAP

Where to work next.

Prioritized by how directly each gap affects answer reliability and supported question types.

First	**Run the full 50-question gold evaluation.** Save the answers, metrics, model, date, and configuration. This establishes a real baseline.
First	**Make table headers machine-readable.** Improve multi-row/grouped header mapping and validate the requested row/column intersection.
Next	**Measure retrieval Recall@5.** Record when the gold page is missing from Gemini's input, then improve ranking/context based on misses.
Next	**Strengthen OCR evidence.** Detect low-quality OCR and avoid confident numeric answers from ambiguous digits/layout.
Later	**Add visual questions deliberately.** Send relevant rendered pages as image input and expand the answer/validator contract for descriptions and names.
Later	**Calibrate confidence and control latency.** Use held-out results, record request timings/retries, and tune timeouts against actual measurements.

Known manual example: the local report's page 241 has a Kollam two-wheeler row with 124 persons killed in 2023. An earlier run abstained despite the evidence. Treat this as a known retrieval/table-header failure; the whole-dataset impact has not been measured.

OPERATE

Run, configure, and score.

All credentials stay in a local ignored `.env`. Do not share a trace without reviewing the page text it contains.

Local environment

```
LLM_API_KEY=<your Gemini API key>
LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai/
LLM_MODEL=gemini-3.8-flash
LLM_TIMEOUT_S=180
LLM_CONTEXT_PAGES=5
RETRIEVAL_TOP_PAGES=5
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
HF_HUB_OFFLINE=1
```

The source defaults use Groq if no local override exists. Hugging Face offline mode and `local_files_only=True` keep embedding loads on the local model cache.

Start the Streamlit UI

```
python -m venv .venv
.venv\\Scripts\\Activate.ps1
python -m pip install -r requirements.txt
# Optional local embeddings and OCR dependencies
```

```
python -m pip install -r requirements-optional.txt
streamlit run app.py
```

Run and score the public example batch

```
python run_batch.py --pdf dataset\\report_A.pdf `
  --questions dataset\\questions_with_answers.jsonl --out answers.jsonl
python evaluate.py --answers answers.jsonl `
  --gold dataset\\questions_with_answers.jsonl --out metrics.json
```

> With a key configured, this sends report-page text to Gemini and uses provider quota. An already ingested PDF may be reused; pass `--force` to re-ingest it.

SOURCE

Code map.

Start with the orchestrators, then follow a single responsibility into its module.

PATH	WHAT IT OWNS
`app.py`	Streamlit layout, chat messages, source display, request trace.
`ingest_service.py`	PDF ingestion orchestration, reuse by hash, progress reporting.
`pdf_loader.py`	Coordinate-sorted text and rendered PDF pages.
`ocr.py · table_extract.py`	Scanned-page OCR and row/header extraction.
`answerer.py`	End-to-end answer sequence and abstain/fallback paths.
`hybrid_search.py · cell_lookup.py`	Candidate page retrieval and exact mapped-cell lookup.
`llm_client.py · prompts.py`	Provider HTTP request, trace, JSON prompt, response parsing.
`validator.py · evaluate.py`	Answer/page checks and gold-set scoring.
`db.py · config.py`	SQLite data access and environment-based settings.

Source basis: Report Navigator repository at HEAD `c697102`, with the local report database inspected on 2026-10-31. Ingestion counts describe that local database and may change after re-ingestion.
